



AMRITA
VISHWA VIDYAPEETHAM

School of Engineering
Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

WORKSHEET

- 1

Name : Ganath Avinash G R

Roll No : CH.SC.U4CSE24118

Section : B



WORKSHEET-1

EX.NO : 1

DATE : 22/06/2026

TITLE: INTRODUCTION TO BITS, GATES AND QUANTUM CIRCUITS

AIM:

To study the fundamental concepts of quantum computing, including classical bits, quantum bits (qubits), quantum states, superposition, quantum gates, and quantum circuit representation. The objective is to understand how quantum information is represented and manipulated using various quantum gates and circuit diagrams, which form the foundation of quantum computation.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit Quantum Computing Framework
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

INSTALLATION:

- Step 1: Install Python
- Download and install Python 3.10 or later from the official Python website.
- During installation, select the option "Add Python to PATH".
- Step 2: Verify Python Installation
- Open Command Prompt and execute:
- `python --version`
- The installed Python version should be displayed.



- Step 3: Create a Virtual Environment
- Create a new virtual environment for Qiskit:
- `python -m venv qiskit-env`
- Activate the environment:
- `qiskit-env\Scripts\activate`
- Step 4: Upgrade pip
- Upgrade the package installer using:
- `python -m pip install --upgrade pip`
- Step 5: Install Qiskit
- Install the Qiskit framework using:
- `pip install qiskit`
- Step 6: Install Qiskit Aer Simulator
- Install the Aer simulator package:
- `pip install qiskit-aer`
- Step 7: Verify Qiskit Installation
- Execute the following command:
- `python -c "import qiskit; print(qiskit.__version__)"`
- If the Qiskit version is displayed, the installation is successful.
- Step 8: Test Qiskit
- Run the following Python code:
- `from qiskit import QuantumCircuit`
- `qc = QuantumCircuit(2)`
- `qc.h(0)`
- `qc.cx(0,1)`
- `print(qc)`
- The quantum circuit diagram will be displayed, confirming successful installation and



execution of Qiskit.

THEORY:

Introduction to Quantum Computing:

Quantum computing is a revolutionary computing paradigm that leverages the principles of quantum mechanics to process information. Unlike classical computers that operate using bits, quantum computers utilize quantum bits (qubits), enabling enhanced computational capabilities for specific classes of problems.

Classical Bits and Quantum Bits:

A classical bit can exist only in one of two possible states, namely 0 or 1. In contrast, a qubit can exist in the state $|0\rangle$, $|1\rangle$, or a superposition of both states simultaneously, allowing quantum systems to represent and process information more efficiently.

Mathematical Representation of a Qubit:

The state of a qubit is represented as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β are complex probability amplitudes satisfying the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

Quantum Circuit Model:

Quantum computations are represented using quantum circuits. A quantum circuit consists of qubits, quantum gates, and measurement operations. Information flows from left to right, with quantum gates manipulating the states of qubits throughout the computation process.

Quantum Gates:

Quantum gates are reversible operations that alter the state of qubits. They serve as the fundamental building blocks of quantum algorithms. Commonly used quantum gates include:

- Pauli-X Gate (X)
- Pauli-Y Gate (Y)
- Pauli-Z Gate (Z)
- Hadamard Gate (H)
- Phase Gate (S)



- T Gate
- Controlled-NOT Gate (CNOT)
- Quantum Superposition

Superposition is a fundamental property of quantum systems that allows a qubit to exist in multiple states simultaneously. The Hadamard gate is commonly used to generate superposition states from computational basis states.

Quantum Entanglement:

Entanglement is a unique quantum phenomenon in which two or more qubits become correlated such that the state of one qubit depends on the state of another, regardless of the distance between them. The CNOT gate is frequently used to create entangled states.

Qiskit Framework:

Qiskit is an open-source quantum computing software development framework developed by IBM. It enables users to design, simulate, visualize, and execute quantum circuits on both simulators and real quantum hardware.

Quantum Circuit Simulation:

Qiskit provides simulation tools such as Aer Simulator for testing and validating quantum circuits in a classical computing environment before deploying them on actual quantum processors.

Measurement of Quantum States:

Measurement is the process of converting quantum information into classical information. Upon measurement, a qubit collapses to one of its basis states, and the outcome is recorded as classical data.

Significance of the Experiment

The study of bits, qubits, quantum gates, and quantum circuits using Qiskit provides the fundamental knowledge required for understanding advanced quantum algorithms and quantum information processing systems.

PROCEDURE:

1. Install Python software and configure the Python environment.
2. Install the Qiskit framework and required dependencies.
3. Verify the successful installation of Qiskit.



4. Open Jupyter Notebook or Visual Studio Code.
5. Import the required Qiskit modules.
6. Create a quantum circuit using the QuantumCircuit class.
7. Initialize qubits and classical bits in the circuit.
8. Apply basic quantum gates such as Pauli-X and Hadamard gates.
9. Construct a simple quantum circuit and visualize it.
10. Execute the quantum circuit using the Aer Simulator.
11. Measure the qubits and obtain the output results.
12. Observe and analyze the behavior of bits, qubits, gates, and quantum circuits.

PROGRAM:

1. Create a quantum circuit with a single qubit and display the circuit diagram.
2. Write a Qiskit program to apply the Pauli-X (NOT) gate on a qubit and observe the circuit.
3. Create a quantum circuit using the Hadamard gate and study the concept of superposition.
4. Write a Qiskit program to create a two-qubit circuit and apply a Controlled-NOT (CNOT) gate.
5. Construct a Bell State circuit using Hadamard and CNOT gates.
6. Write a Qiskit program to measure a single qubit and display the output.
7. Create a quantum circuit with two qubits and perform measurements on both qubits.
8. Design and visualize a quantum circuit containing X, H, and CNOT gates.
9. Simulate a quantum circuit using the Aer Simulator and analyze the measurement results.
10. Compare the behavior of classical bits and quantum bits using simple Qiskit circuits.

OUTPUT:

1. Create a quantum circuit with a single qubit and display the circuit diagram.

Aim:

To create a single-qubit circuit and study the X gate, Hadamard gate, and superposition using Qiskit.

Code:

```
[8]: # Q1
from qiskit import QuantumCircuit

# Create a quantum circuit with 1 qubit
qc = QuantumCircuit(1)

# Display the circuit
print(qc)
```

Output:

q:

2. Write a Qiskit program to apply the Pauli-X (NOT) gate on a qubit and observe the circuit.

Aim:

To create two-qubit circuits, implement CNOT and Bell states, and study entanglement and measurement.

Code:

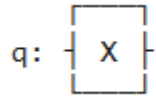
```
[9]: # Q2
from qiskit import QuantumCircuit

# Create a circuit with 1 qubit
qc = QuantumCircuit(1)

# Apply Pauli-X gate
qc.x(0)

# Display the circuit
print(qc)
```

Output:



3. Create a quantum circuit using the Hadamard gate and study the concept of superposition.

Aim:

To design and simulate quantum circuits using **Aer Simulator** and compare **classical bits with qubits**.

Code:

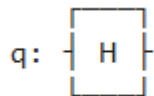
```
# Q3
from qiskit import QuantumCircuit

# Create a circuit with 1 qubit
qc = QuantumCircuit(1)

# Apply Hadamard gate
qc.h(0)

# Display the circuit
print(qc)
```

Output:



Measurement & Series:

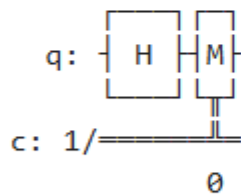
```
# Q1 , Q2 With Measurement
from qiskit import QuantumCircuit

# Create 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)

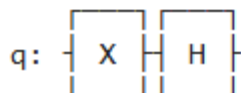
# Apply Hadamard gate
qc.h(0)

# Measure qubit
qc.measure(0, 0)

# Display circuit
print(qc)
```



```
# Set of major commands used
qc = QuantumCircuit(1) # Create 1 qubit
qc.x(0)                 # Pauli-X / NOT gate
qc.h(0)                 # Hadamard gate
print(qc)               # Display circuit
```



4. Write a Qiskit program to create a two-qubit circuit and apply a Controlled-NOT (CNOT) gate.

Aim:

To create a two-qubit quantum circuit and apply a Controlled-NOT (CNOT) gate.

Code:

```
# Q4
from qiskit import QuantumCircuit

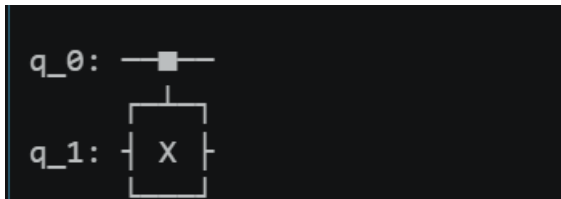
qc = QuantumCircuit(2)

qc.cx(0, 1)

print(qc)
```

✓ 2.2s

Output:



5. Construct a Bell State circuit using Hadamard and CNOT gates.

Aim:

To construct a Bell State using Hadamard and CNOT gates.

Code:

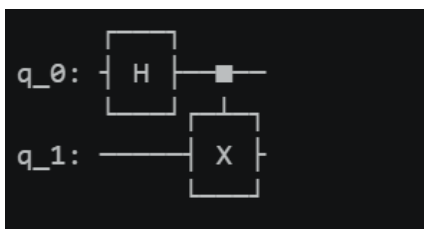
```
# Q5
from qiskit import QuantumCircuit

qc = QuantumCircuit(2)

qc.h(0)
qc.cx(0, 1)

print(qc)
```

Output:



6. Write a Qiskit program to measure a single qubit and display the output.

Aim:

To create a quantum circuit, measure a single qubit, and display the measurement output.

Code:

```
# Q6
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

qc = QuantumCircuit(1, 1)

qc.h(0)
qc.measure(0, 0)

simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

print(result.get_counts())
```

✓ 0.3s

Output:

```
{'1': 525, '0': 499}
```

7. Create a quantum circuit with two qubits and perform measurements on both qubits.

Aim:

To create a two-qubit quantum circuit and perform measurements on both qubits.

Code:

```
# Q7
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

qc = QuantumCircuit(2, 2)

qc.h(0)
qc.h(1)

qc.measure(0, 0)
qc.measure(1, 1)

simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

print(result.get_counts())
```

Output:

```
{'11': 279, '10': 249, '01': 260, '00': 236}
```

8. Design and visualize a quantum circuit containing X, H, and CNOT gates.

Aim:

To design and visualize a quantum circuit containing X, H, and CNOT gates.

Code:

```
# Q8
from qiskit import QuantumCircuit

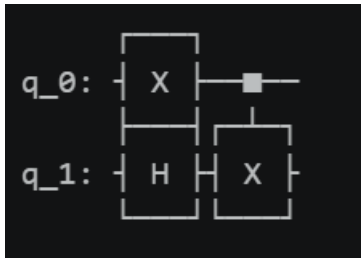
qc = QuantumCircuit(2)

qc.x(0)
qc.h(1)
qc.cx(0, 1)

print(qc)

✓ 0.0s
```

Output:



9. Simulate a quantum circuit using the Aer Simulator and analyze the measurement results.

Aim:

To simulate a quantum circuit using the Aer Simulator and analyze the measurement results.

Code:

```
# Q9
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

qc = QuantumCircuit(2, 2)

qc.h(0)
qc.cx(0, 1)

qc.measure(0, 0)
qc.measure(1, 1)

simulator = AerSimulator()

result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()

print("Measurement Results:", counts)
```

Output:

```
Measurement Results: {'11': 491, '00': 533}
```

10. Compare the behavior of classical bits and quantum bits using simple Qiskit circuits.

Aim:

To compare the behavior of classical bits and quantum bits using simple Qiskit circuits.

Code:

```
# Q10
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

# Classical bit: definite value
classical = QuantumCircuit(1, 1)
classical.measure(0, 0)

# Quantum bit: superposition
quantum = QuantumCircuit(1, 1)
quantum.h(0)
quantum.measure(0, 0)

simulator = AerSimulator()

print("Classical bit:")
print(simulator.run(classical, shots=10).result().get_counts())

print("Quantum bit:")
print(simulator.run(quantum, shots=1000).result().get_counts())
```

Output:

```
Classical bit:
{'0': 10}
Quantum bit:
{'0': 529, '1': 471}
```



1. What is Quantum Computing? (VIVA QNS)

Quantum computing is a type of computing that uses **qubits and quantum principles** to process information.

2. What is the difference between a classical bit and a qubit?

Difference between classical bit and qubit

- **Bit:** Can be either 0 or 1.
- **Qubit:** Can be in 0, 1, or a **superposition of both**.

3. What is superposition in quantum computing?

Superposition means a qubit can exist in a combination of $|0\rangle$ and $|1\rangle$ simultaneously until measured

4. Write the mathematical representation of a qubit.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

5. What is the normalization condition of a qubit?

- $|\alpha|^2 + |\beta|^2 = 1$

6. What is the function of the Pauli-X gate?

The Pauli-X gate **flips the qubit**:

$|0\rangle \rightarrow |1\rangle$ and $|1\rangle \rightarrow |0\rangle$.

It is similar to a **NOT gate**.

7. What is the purpose of the Hadamard gate?

It creates **superposition**.

$$|0\rangle \rightarrow (|0\rangle + |1\rangle)/\sqrt{2}$$

8. What is a Controlled-NOT (CNOT) gate?

CNOT is a two-qubit controlled gate. It flips the target qubit only when the control qubit is 1

9. What is quantum entanglement?

Entanglement is a quantum phenomenon where two or more qubits become correlated, so the state of one is related to the state of the other, even when separated.

10. What is Qiskit and why is it used in quantum computing?

Qiskit is an open-source Python framework for quantum computing. It is used to create, simulate, and run quantum circuits on quantum computers.

RESULT:

The quantum circuits were successfully created and simulated using Qiskit. The X, H, and CNOT gates were implemented and their behavior was observed. Superposition and entanglement were demonstrated using H and CNOT gates. Single- and two-qubit measurements were performed, and the results were analyzed using the Aer Simulator. The difference between classical bits and qubits was also observed.